

ProofPatch

Verify the fix. Before you ship.

Reproducible verification for AI-assisted code repairs

IBM Bob 2.0 Hackathon · Fix bugs. Show proof.

Mohammad Aazam · Mirza Yasir Abdullah Baig · Faza E Badar

A missing record can go unnoticed

A convincing patch still needs a test that catches the failure.

Original: task 4 disappears at a page boundary

1 2 3 — 5 6 7 8

Repair: every task appears exactly once

1 2 3 4 5 6 7 8

Our IBM Bob workflow

Reproduce → preserve tests → repair → verify → share evidence

We used Bob in our original development

Our reusable local skill gives the repair workflow a repeatable structure.

After using our credits, we continued local improvements and testing.

Our verification contract

A repair earns acceptance only against the preserved test suite.

- 01 Check the protected baseline, fixture and tests
- 02 Reproduce the named original assertion failure
- 03 Run the identical complete suite on the repair
- 04 Export the exact executed code and evidence

Results we can reproduce

One synthetic case, tested through success and failure.

30 / 30

Original defect: reproduced

Incorrect repair: rejected

candidate behavior checks pass

35 verifier + interface checks pass; exported snapshot reruns successfully

Who benefits

Reviewable evidence connects AI development to accountable release decisions.

Developers + reviewers

One repeatable handoff: defect, patch, tests and logs.

IBM's developer ecosystem

A reusable verification pattern around Bob-assisted work.

Governance: traceability • End users: fewer missed defects

Our first users and product path

Start with small Python teams reviewing AI-assisted changes.

Broad market: teams adopting AI coding tools

Initial segment: Python teams with pytest workflows

Our focus: reproducible evidence in one review package

Pilot next: repository + CI adapters; measure review effort

Revenue hypothesis: hosted team history and CI reports

Verify the fix. Before you ship.

ProofPatch connects AI-assisted repairs to evidence reviewers can check.

Reproduce the defect. Verify the repair. Share the run.

Try all three outcomes and independently rerun the evidence.

github.com/aazamcs/proofpatch